



Project Report

On

HUMAN POSE DETECTION FOR CHEATING PREVENTION IN EXAMS

*Submitted in partial fulfillment of the requirements
for the award of the degree of*

BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

Vikas Kumar
2022BITE077

Ayasha Neeraj
2022BITE080

Under the supervision of

Dr. Arooj Nissar

Department of Information Technology
National Institute of Technology Srinagar

June 2026



CERTIFICATE

This is to certify that the project titled **Human Pose Detection For Cheating Prevention in Exams** has been completed by **Vikas Kumar (2022BITE077)** and **Ayasha Neeraj (2022BITE080)** under my supervision in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Information Technology**. It is also certified that the project has not been submitted for the award of any other degree.

Dr. Arooj Nissar

Supervisor

Department of Information Technology

NIT Srinagar



STUDENTS' DECLARATION

We hereby declare that the work presented in this project entitled **Human Pose Detection For Cheating Prevention in Exams** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Information Technology**, is an authentic record of our own work carried out under the supervision of **Dr. Arooj Nissar**, Department of Information Technology, NIT Srinagar. This project has not been submitted for the award of any other degree.

Dated: _____

(i) Name: **Vikas Kumar**

Signature: _____

(ii) Name: **Ayasha Neeraj**

Signature: _____

ACKNOWLEDGEMENT

We sincerely thank our project supervisor, **Mrs. Arooj Nissar**, for her constant guidance, support, and encouragement throughout the course of this project. Her insights, valuable suggestions, and constructive feedback helped us overcome challenges and successfully complete this work on **Human Pose Detection For Cheating Prevention in Exams**.

We are also grateful to all the faculty members of the Department of Information Technology for their guidance and support whenever we needed it. Their encouragement and expertise greatly contributed to our learning experience.

Finally, we extend our heartfelt thanks to our friends and families, whose motivation, understanding, and support kept us inspired throughout this project.

Vikas Kumar (IT/77/22)
Ayasha Neeraj (IT/80/22)

Abstract

It is becoming increasingly difficult to maintain integrity during exams, with the increase in online exams and camera exams. Although manual invigilation helps guard against cheating, this process can be tiring for the invigilator, as well as time-consuming and arduous when there are so many students being invigilated at the same time.

The **Human Pose Detection for Cheating Prevention in Exams** project has developed a computer vision-based tool to help detect suspicious behaviour during examinations. When the examination is conducted with an uploaded video or webcam input, the software evaluates the posture of the student's body; head movements; visibility of their face; position of their hands; and whether there are any objects such as mobile telephones, books, or chits present and accessible to the student during the examination. Instead of relying on just one frame to determine whether the student's behaviour is suspicious, the system will track the student's behaviour throughout multiple frames (i.e., a sequence of frames) to make the classification of the student's behaviour more reliable.

After all the required features are captured, they are passed to a Bi-LSTM model that determines whether the student's behaviour is normal or suspicious, as well as providing a reason as to why the behaviour was deemed to be suspicious, and an event log with the timestamps of where the behaviour was suspicious or irregular.

The Human Pose Detection for Cheating Prevention in Exams project demonstrates that deep learning and human pose detection can assist in monitoring examinations, with the intent that the system will assist and not replace invigilators.

Contents

List of Figures	vii
List of Tables	viii
Abbreviations	ix
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	1
2 Literature Survey	2
2.1 Literature Review	2
3 Human Pose Detection	6
3.1 Head Pose	6
3.2 Eye Direction	6
3.3 Shoulder Position	6
3.4 Body Leaning and Posture	7
3.5 Role of Pose Detection in the Proposed System	7
4 Bi-LSTM, MediaPipe and Yolo	8
4.1 Bidirectional Long Short-Term Memory	8
4.1.1 Framework	8
4.1.2 Applications	9
4.2 MediaPipe	9
4.2.1 Framework	9
4.2.2 Applications	10
4.3 YOLO (You Only Look Once)	10
4.3.1 Framework	10
4.3.2 Applications	11
5 Implementation and Results	12
5.1 Framework	13
5.2 Dataset Collection	14
5.3 OpenCV Frame Reader	14
5.4 Feature Extraction	15
5.5 Identification of Pose and Object	16
5.6 From Landmarks to Behavior Features	17
5.7 Feature Fusion	18

5.8	Bi-LSTM Architecture Implementation	19
5.9	Main System Layers	20
5.10	Training	20
5.11	Testing	21
5.12	Threshold Tuning and Runtime Logic	21
5.13	Frontend and Backend	23
5.14	Results	24
6	Discussion	25
7	Conclusion and Future Work	28
7.1	Conclusion	28
7.2	Future Work	28
	Bibliography	30
A	Software and Hardware Requirements	34
A.1	Software Requirements	34
A.2	Hardware Requirements	34
B	Important Code Snippets	35
B.1	Bi-LSTM Model	35
B.2	Final Decision Rule	35
C	Project Commands	36

List of Figures

4.1	Bi-Lstm Framework	8
4.2	MediaPipe Framework	9
4.3	YOLO Framework	10
5.1	Framework for Cheating Prevention in Exams	13
5.2	Distribution of normal and suspicious videos dataset	14
5.3	Sequence windowing	18
5.4	Bi-LSTM architecture	19
5.5	Decision Logic: Model Probability + Rule based evidence	22
5.6	Bi-LSTM train test confusion matrices	24

List of Tables

2.1	Literature Survey of Existing Exam Cheating Detection Systems	4
5.1	Dataset and Feature Summary	15
5.2	Important implementation files used in the system	16
5.3	Implemented Bi-LSTM architecture	19
5.4	Main layers of the implemented system	20
5.5	Training configuration used in the implemented model	21
5.6	Important backend routes	23
5.7	Train and Test Comparison	24
6.1	Final Model Details	25
6.2	64-unit and 128-unit Bi-LSTM Comparison	26
6.3	Sample prediction outcomes	27
A.1	Software requirements	34

Abbreviations

AI	Artificial Intelligence
Bi-LSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
CPU	Central Processing Unit
FPS	Frames Per Second
GUI	Graphical User Interface
ML	Machine Learning
NIT	National Institute of Technology
ROI	Region of Interest
YOLO	You Only Look Once

Chapter 1

Introduction

1.1 Background

The COVID-19 pandemic caused a major shift in the education system, with examinations moving from offline to online mode. Although online exams provide convenience, they also increase the risk of unfair practices. Therefore, there is a growing need for cheating detection systems that can help maintain the integrity and fairness of online examinations [1].

In traditional exams, invigilators monitor students to prevent cheating. However, in online examinations, manual monitoring can be difficult. An automated cheating detection system can identify suspicious activities such as looking away from the screen, using mobile phones, books, or notes, and the presence of multiple persons during the exam. Previous studies have shown that computer vision and deep learning techniques can be used effectively for this purpose [2].

1.2 Problem Statement

To develop an exam monitoring system that extracts visual behavior features from the examination videos and uses temporal sequence analysis to classify behavior as normal or suspicious. The system should analyze repeated behavior patterns over time and identify the normal or suspicious behavior.

1.3 Objectives

This project aims to:

1. The first objective is dataset collection.
2. Implementing the Bi-LSTM model
3. Training and Testing of the model
4. Real time evaluation using UI

Chapter 2

Literature Survey

A lot of research work has been carried out in the field of cheating detection for online examinations, especially using human pose detection and computer vision techniques. The literature survey helped us understand the different methods used by researchers, along with their advantages, limitations, and possible areas for improvement. Many studies have focused on monitoring student behavior during online exams by analyzing activities such as head movements, gaze direction, body posture, and the use of unauthorized objects like mobile phones, books, and notes. Researchers have used techniques such as gaze estimation, head pose tracking, human pose estimation, and object detection to identify suspicious behavior. Some systems also combine multiple methods to improve detection accuracy and reduce false alarms. With recent advancements in machine learning and deep learning, the performance of these systems has improved significantly. However, challenges such as varying lighting conditions, camera quality, occlusions, and false detections still exist. Therefore, there is a need for more accurate and reliable cheating detection systems that can effectively monitor online examinations.

2.1 Literature Review

In 2020, Zhang and Wang [29] proposed a video-based examination surveillance system to improve academic integrity. The system used computer vision techniques and automated monitoring to observe students during examinations. Their approach reduced human monitoring effort and improved examination security. However, the system mainly focused on video surveillance and lacked advanced behavioral analysis capabilities.

In 2021, Cheng et al. [30] developed an automated online examination monitoring framework using computer vision technology. The system continuously monitored candidates and detected suspicious activities in real time. The study demonstrated the effectiveness of automated monitoring for large-scale online examinations. However, the framework was sensitive to lighting conditions and camera quality.

In 2021, Hossain et al. [31] introduced a visual behavior analysis framework for online examinations. Their method analyzed head movements, eye gaze direction, and speaking activities to identify suspicious behavior. Experimental results showed improved cheating detection accuracy. However, the system generated false alarms when students naturally moved during examinations.

In 2021, Kamalov et al. [32] proposed an AI-based online proctoring system using facial behavior analysis and deep learning algorithms. The framework employed facial

recognition and behavior tracking techniques to monitor examinees. The study demonstrated that computer vision methods can effectively support remote examination integrity. Nevertheless, privacy concerns and computational requirements remained major challenges.

In 2022, Abozaid and Atia [33] developed a multimodal online cheating detection system that combined face detection, facial landmark extraction, eye tracking, and head pose estimation. The integration of multiple monitoring techniques improved the accuracy of suspicious activity detection compared to single-feature approaches. However, the system required high computational resources for real-time implementation.

In 2023, Khan et al. [34] proposed a deep learning based cheating detection framework for online examinations. Their system utilized face recognition, eye tracking, object detection, and behavior analysis to identify cheating activities. The results showed better detection accuracy through multimodal monitoring. However, performance decreased in complex environments with multiple candidates.

In 2024, Bishaw et al. [35] proposed a deep learning model for paper-based examination cheating detection. The study applied CNN, AlexNet, GoogLeNet, and ResNet architectures on cheating and non-cheating image datasets. Their customized CNN model achieved an accuracy of 93%. However, the dataset size was limited and did not cover all cheating scenarios.

In 2024, Shiri et al. [36] performed a comparative analysis of different deep learning architectures for online exam cheating detection. The study evaluated multiple transfer learning models and compared their performance. Results showed that transfer learning approaches achieved higher accuracy and efficiency than traditional machine learning methods. However, model performance depended heavily on the quality of training data.

In 2025, Rahmawati et al. [39] proposed a CNN-based cheating detection system using eye gaze tracking, head pose estimation, and lip movement analysis. The system monitored students through webcams and detected suspicious behavior in real time. Experimental results demonstrated reliable detection performance for online examinations. However, the approach mainly focused on facial features and ignored object-based cheating activities.

In 2025, Abourifa et al. [38] introduced a transfer learning based cheating detection system using the MobileNetV2 architecture. Their model analyzed candidate behavior, mobile phone usage, unauthorized persons, and cheating-related gestures. The proposed framework achieved an accuracy of 98.3%, outperforming several existing models. However, the model required a large amount of labeled training data.

In 2025, Abdulkarim et al. [37] developed a real-time AI-driven cheating detection system using YOLOv8 object detection and OpenPose human pose estimation. The multimodal framework detected prohibited objects such as mobile phones and suspicious body movements simultaneously. The system achieved high precision and real-time performance. However, accuracy could be affected by camera positioning and occlusion problems.

In 2025, Dahan et al. [40] presented a comprehensive review of AI-driven approaches for online exam integrity. The study analyzed deep learning models, multimodal monitoring systems, and real-time proctoring technologies. The authors highlighted the advantages of combining visual, audio, and behavioral features for cheating detection. However, they identified privacy concerns, scalability issues, and high computational costs as major limitations of existing systems.

Research Paper (Year)	Area / Topic	Model / Technology Used	Performance / Limitation
Real-Time AI-Driven Exam Cheating Detection Using YOLO and Pose Estimation: A Multi-Modal Deep Learning Approach (2025)	Pose + Object Detection	YOLOv8 object detection with pose-based behavioral analysis.	Precision: 0.92, Recall: 0.88, F1-score: 0.90. Lacks long-duration temporal behavior analysis.
Multiple Instance Learning for Cheating Detection and Localization in Online Examinations (2024)	CHEESE Framework	Eye gaze, head posture, facial features, body posture, and background feature fusion.	Comprehensive approach, but multi-person gaze correlation remains unexplored.
Deep Learning Based multimodal cheating detection in online proctored exams (2024)	Gaze +pose + object detection	Based on CNN, L2CSNet	87% accuracy. Decisions are made from individual images temporal context.
Multi-Modal Online Exam Cheating Detection (2022)	Multi-Modal Online Proctoring	Combines head-pose detection, object detection, and eye-gaze estimation using webcam-based monitoring.	Reported event detection accuracy: 95.69%; tested on 29 students across four exam sessions.
Detection of Malpractice in E-exams by Head Pose and Gaze Estimation (2021)	2D Gaze + Head Pose	Head pose estimation, eye gaze tracking, and Visual Focus of Attention (VFOA).	96% accuracy. Limited focus on physical classroom examinations.
Gaze360: Physically Unconstrained Gaze Estimation in the Wild (2019)	3D Gaze Estimation	Large-scale 3D gaze dataset and deep gaze estimation framework.	Performance affected by lighting conditions, glasses, and subject variations.
Appearance-Based Gaze Estimation in the Wild (2015)	3D Gaze Tracking	Appearance-based gaze estimation using deep learning.	Requires calibration for each individual; not scalable for large examination halls.

Table 2.1: Literature Survey of Existing Exam Cheating Detection Systems

From the literature survey, we conclude that a significant amount of research has been carried out in the field of online exam cheating detection using computer vision, machine learning, and deep learning techniques. Researchers have used various approaches such as gaze estimation, head pose detection, human pose estimation, facial landmark analysis, and object detection to identify suspicious activities during examinations. Technologies such as MediaPipe, YOLO, OpenPose, CNNs, and deep learning-based gaze tracking models have shown promising results in detecting behaviors like looking away from the screen, using unauthorized materials, and the presence of multiple persons. The survey also shows that combining multiple detection techniques generally provides better accuracy and reliability than using a single method. However, existing systems still face challenges such as false alarms, limited dataset diversity, varying lighting conditions, camera quality issues, and privacy concerns. Therefore, there is a need for a more robust and efficient cheating detection system that can accurately monitor candidate behavior in real time while minimizing incorrect detections.

Chapter 3

Human Pose Detection

Human Pose Detection is a computer vision technique used to identify and track the position of different parts of the human body in images and videos. It works by detecting key body landmarks, such as the head, eyes, shoulders, elbows, wrists, hips, knees, and ankles, and connecting them to form a skeletal representation of the person. This representation helps in understanding a person's posture, body movements, and actions without requiring detailed information about appearance or clothing. Human pose detection has applications in various fields, including fitness tracking, healthcare, sports analysis, gaming, animation, human-computer interaction, and surveillance. With recent advancements in deep learning, pose estimation models have become more accurate and capable of working in real-time environments.

3.1 Head Pose

The position and orientation of the head provide important information about a person's attention and focus. By analyzing head landmarks, it is possible to estimate whether the head is facing forward, tilted, or turned to the side. In our work, head pose information is used to identify repeated side-looking or downward-looking behavior that may indicate suspicious activity during an examination.

3.2 Eye Direction

Eye-related landmarks help determine the direction of a person's gaze. Gaze estimation can indicate whether an individual is looking at the screen or repeatedly looking elsewhere. In our system, eye direction analysis supports the detection of attention shifts and unusual viewing patterns during the examination.

3.3 Shoulder Position

The position of the shoulders helps in understanding upper-body posture. Stable shoulder alignment generally indicates a normal sitting position, while significant movement or uneven shoulder orientation may suggest unusual body activity. In our work, shoulder landmarks contribute to posture analysis and movement tracking.

3.4 Body Leaning and Posture

Human pose detection enables the analysis of overall body posture by tracking the relative positions of key body landmarks. Leaning excessively to one side, moving out of the camera frame, or maintaining an unusual posture for long periods can be detected through pose information. In our system, body posture analysis is used to identify suspicious movements that may not be visible through face analysis alone.

3.5 Role of Pose Detection in the Proposed System

Human pose detection forms an important component of the proposed cheating detection system. It provides body landmark coordinates from video frames and helps monitor posture, head movement, and body orientation over time. These pose-based features are combined with facial and object-related information to improve the detection of suspicious behaviors. The extracted pose sequences are further utilized by the Bi-LSTM model to analyze temporal movement patterns and distinguish between normal and suspicious activities.

Chapter 4

Bi-LSTM, MediaPipe and Yolo

4.1 Bidirectional Long Short-Term Memory

A **Bidirectional Long Short-Term Memory (Bi-LSTM)** is a type of recurrent neural network (RNN) that processes sequential data in **both forward and backward** directions simultaneously. Unlike a standard LSTM that reads a sequence only from left to right, a Bi-LSTM maintains two hidden states — one moving forward through time and one moving backward — then combines them to produce a richer contextual representation. This allows the model to capture dependencies from both past and future time steps, making it especially powerful for tasks where meaning depends on surrounding context rather than just prior history.

4.1.1 Framework

The Bi-LSTM architecture consists of two parallel LSTM layers processing the same input in opposite directions. The outputs are concatenated at each time step to form the final representation.

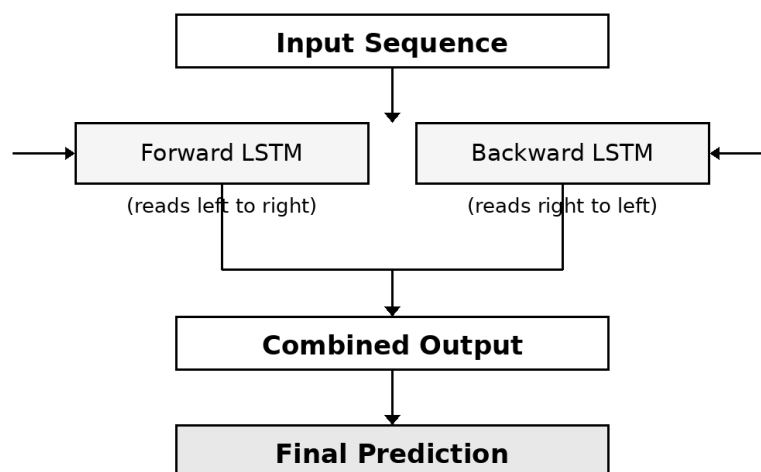


Figure 4.1: Bi-Lstm Framework

4.1.2 Applications

- **Speech Recognition** — Understands spoken words by considering phonemes that come before and after.
- **Sentiment Analysis** — Determines whether a sentence is positive, negative, or neutral by reading the full context.
- **Sign Language Recognition** — Learns temporal gesture sequences to classify hand signs accurately.
- **Medical Text Analysis** — Reads clinical notes to extract diagnoses and conditions with contextual accuracy.

Bi-LSTM is important because it reads data from both forward and backward directions, which helps it understand the full meaning of a sequence

4.2 MediaPipe

MediaPipe is an open-source, cross-platform framework developed by **Google** for building real-time perception pipelines. It provides pre-trained, production-ready machine learning solutions for detecting and tracking body parts such as hands, face, and full-body pose directly from a camera feed or video stream.

4.2.1 Framework

MediaPipe operates as a *graph-based pipeline* where each node performs a specific task such as feature extraction, model inference, or landmark post-processing.

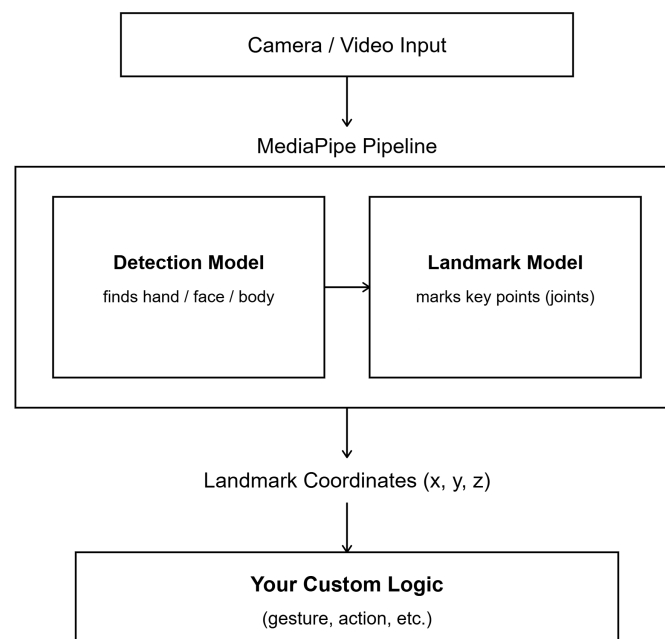


Figure 4.2: MediaPipe Framework

4.2.2 Applications

MediaPipe Pose Detection is used in many areas of daily life. It helps people check their body posture during exercise and yoga. Doctors use it to monitor patient movements during injury recovery. In sports, coaches use it to track player body movements and improve their performance. It can also recognize what a person is doing, like walking or jumping, just by looking at their body. It even helps deaf people by reading hand and body movements to understand sign language.

MediaPipe is important because it gives very accurate key points of the human body like hands, face, and joints in real time.

4.3 YOLO (You Only Look Once)

YOLO (You Only Look Once) is a real-time object detection algorithm that treats detection as a single regression problem. Unlike traditional methods that apply a classifier to multiple regions of an image, YOLO passes the entire image through a single convolutional neural network **just once** and simultaneously predicts bounding boxes and class probabilities for all objects. This makes YOLO extremely fast — capable of processing video frames in real time — while maintaining competitive accuracy, making it one of the most widely used algorithms in computer vision.

4.3.1 Framework

YOLO divides the input image into an $S \times S$ grid. Each grid cell predicts B bounding boxes and C class probabilities.

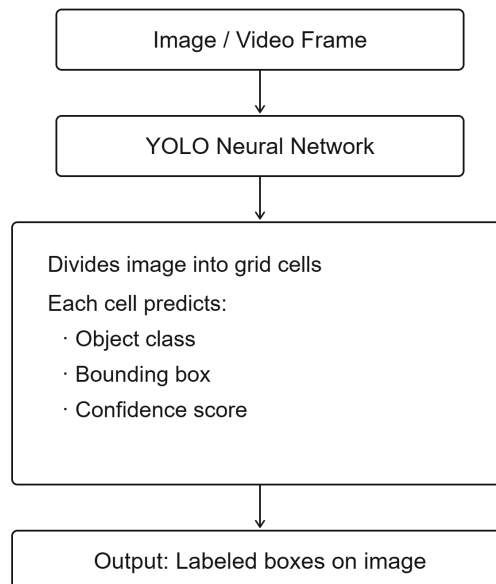


Figure 4.3: YOLO Framework

4.3.2 Applications

YOLO has a various real world applications. It is used to detect human body pose during an exam to find out if a student is cheating or not. It looks at the position of the head, eyes, and body of the student sitting in the exam hall. If a student turns their head too much to the side or looks at another student's paper, YOLO catches that movement instantly. It is also used in security cameras to watch people in public places. In sports, it tracks players and the ball during a match. In hospitals, it helps monitor patient movements. It makes cheating detection faster, more accurate, and does not need a human to watch every student all the time. YOLO is important because it can detect objects and people in a video very fast, in just one single look at the image.

Chapter 5

Implementation and Results

We implemented our proposed work by using a Windows operating system with 16 GB RAM and a standard CPU, which was sufficient for processing video data and running the deep learning models. Python 3.10 was selected as the main programming language because of its simplicity, flexibility, and extensive support for machine learning and computer vision libraries. OpenCV was used to read and process video frames, while MediaPipe was employed to extract face, hand, and pose landmarks from each frame. YOLOv8 was integrated into the system for detecting objects and persons present in the examination environment.

To analyze student behavior over time, the extracted features were combined and provided as input to the Bi-LSTM model. TensorFlow and Keras were used to build, train, and evaluate the deep learning model. NumPy and Pandas were utilized for data preprocessing, feature handling, and result analysis, whereas Scikit-learn was used for performance evaluation. Visual Studio Code served as the development environment for coding and testing the application.

The implementation process involved video frame extraction, feature extraction, object detection, feature fusion, model training, and final prediction generation. The system was tested on examination videos under different scenarios to evaluate its ability to identify suspicious activities. Overall, the selected software tools and hardware resources enabled efficient implementation of the proposed framework and provided reliable performance for cheating detection in examinations.

5.1 Framework

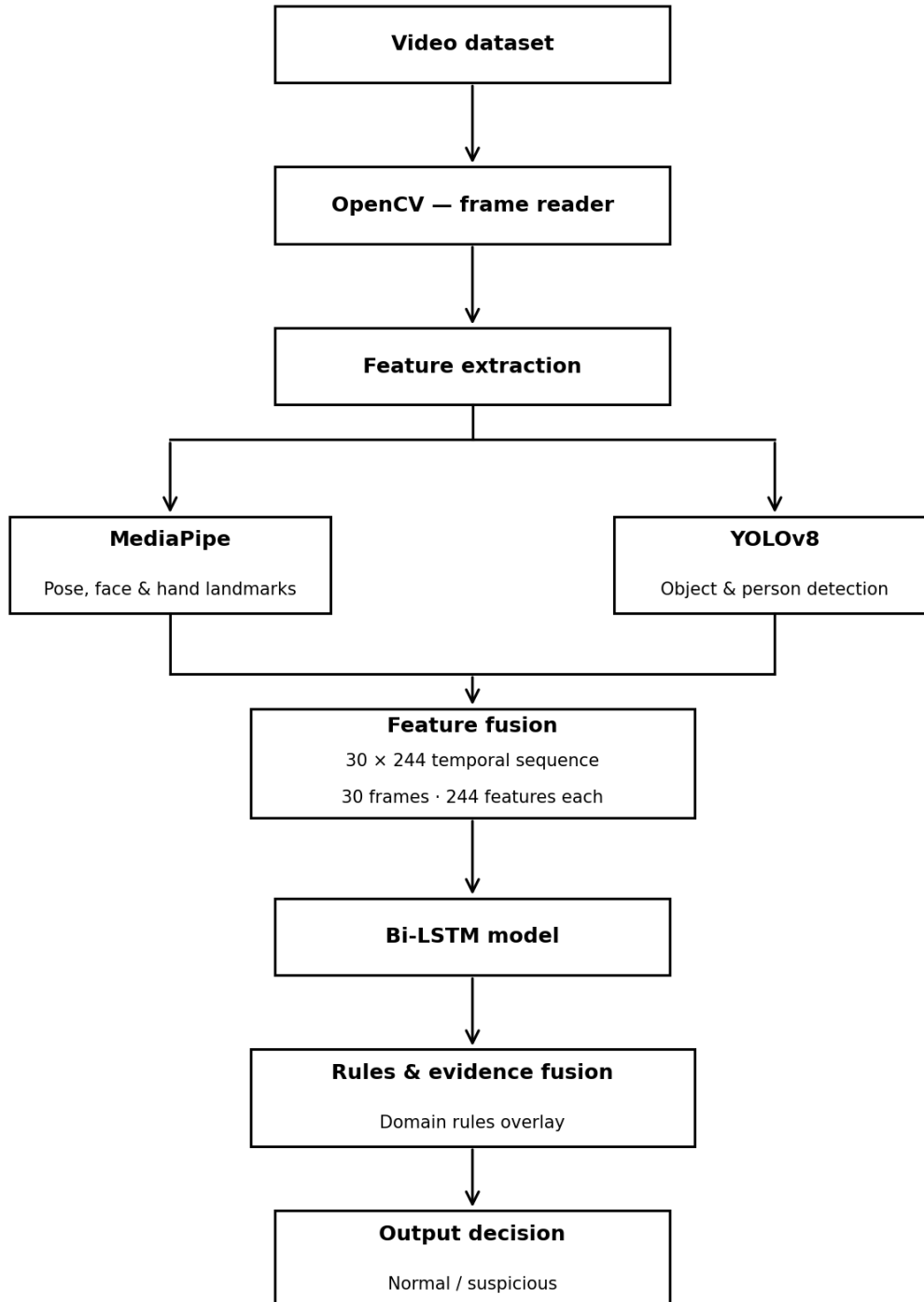


Figure 5.1: Framework for Cheating Prevention in Exams

5.2 Dataset Collection

We collected 102 labeled exam-behaviour videos from our pc webcam, with 51 normal and 51 suspicious cases to keep the dataset balanced. The videos include common exam activities such as writing, reading questions, and looking around naturally. Suspicious samples contain behaviors like repeatedly looking sideways, using hidden notes, passing signals, or unusual movements. This variety helps the model learn to distinguish normal actions from potentially suspicious behavior more accurately.

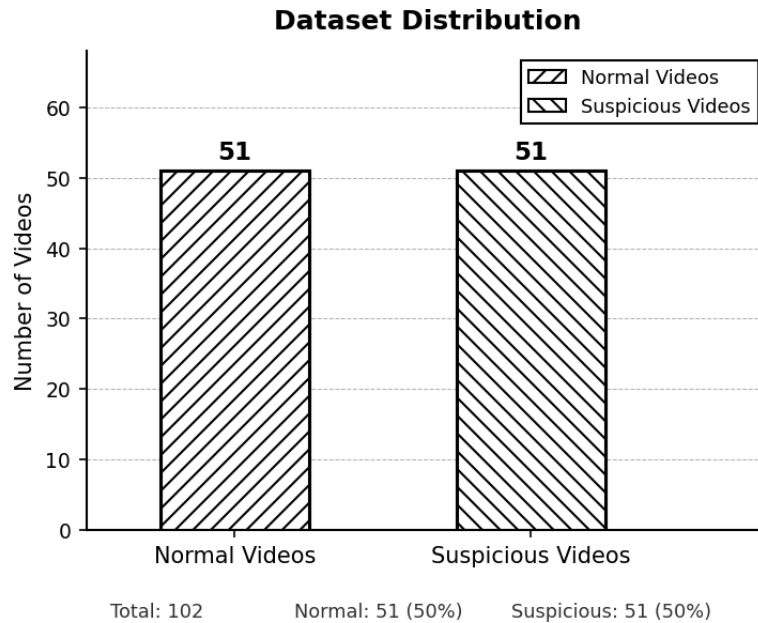


Figure 5.2: Distribution of normal and suspicious videos dataset

5.3 OpenCV Frame Reader

Once the videos were collected and labelled, the next step was to actually read them. OpenCV handles this job. It opens each video file, goes through it frame by frame, and extracts individual images that can then be analysed by the detectors.

However, processing every single frame in a video would be extremely slow and largely unnecessary. If a video runs at 30 frames per second and lasts two minutes, that is 3,600 frames to process. Many of those frames are nearly identical — the student has barely moved from one frame to the next. Processing all of them would waste computing time without adding meaningful new information.

To solve this, the system uses a frame stride of 3. This means it takes one frame, skips the next two, then takes the following one, and so on. Only every third frame is sampled and analysed. This reduces the total number of frames that need to go through the feature extraction pipeline, speeds up both training and inference, and still captures enough visual information to understand what is happening in the video over time. The sampled frames from all 102 videos together produced 31,555 individual frame rows after feature extraction.

Table 5.1: Dataset and Feature Summary

Item	Value
Normal videos	51
Suspicious videos	51
Total videos	102
Sampled feature rows	31,555
Features per frame	244
Total sequences	1,951
Training sequences	1,575
Testing sequences	376
Sequence length	30 frames
Sequence stride	15 frames

5.4 Feature Extraction

The Bi-LSTM model does not see raw video frames. It does not process pixels directly. Instead, every sampled frame is converted into a row of 244 numerical values that describe what was happening in that frame in a structured, compact way. This process is called feature extraction, and it is the most important step in the entire pipeline.

The feature vector for frame t can be written as:

$$\mathbf{x}_t = [\mathbf{b}_t, \mathbf{p}_t, \Delta\mathbf{p}_t]$$

where $\mathbf{b}_t$ contains 13 behaviour and object flags, $\mathbf{p}_t$ contains 132 normalised pose landmark values, and $\Delta\mathbf{p}_t$ contains 99 pose-motion values.

The feature count works out as follows:

$$13 + (33 \times 4) + (33 \times 3) = 244$$

The first group contains 13 behaviour and object flags. These are simple binary values — either 0 or 1 — that answer yes-or-no questions about what is visible in the frame. Is the head turned left? Is the face missing? Is a hand close to the face? Is the body leaning to one side? Is a phone visible? Is a book or chit present? Is there an extra person in the frame? Each of these questions becomes one number in the row.

The second group contains 132 pose landmark values. MediaPipe Pose detects 33 body landmarks in each frame. For each landmark, four values are stored: the normalised x coordinate, the normalised y coordinate, the z coordinate which gives a sense of depth, and a visibility score. This gives:

$$33 \times 4 = 132 \text{ values per frame}$$

Before saving, the coordinates are normalised relative to the centre of the torso and the scale of the body. This means the feature values do not change dramatically just because a student sits closer to the camera one day and farther away the next.

The third group contains 99 pose motion values. For each of the 33 body landmarks, the change in position from the previous sampled frame is calculated:

$$\Delta x_t = x_t - x_{t-1}, \quad \Delta y_t = y_t - y_{t-1}, \quad \Delta z_t = z_t - z_{t-1}$$

This gives:

$$33 \times 3 = 99 \text{ motion values per frame}$$

These motion values are crucial because they tell the model whether the body is moving and how fast. A student who is constantly shifting, turning, or moving hands rapidly is behaving differently from one who is sitting still.

The main implementation files used in the system are shown in Table 5.2.

Table 5.2: Important implementation files used in the system

File	Purpose
<code>feature_extractor.py</code>	Extracts pose, face, hand, motion, and object features from video frames.
<code>sequence_data.py</code>	Converts frame-level rows into fixed-length temporal sequences.
<code>training.py</code>	Builds, trains, tunes, tests, and saves the Bi-LSTM model.
<code>inference.py</code>	Runs prediction on a new video and applies sequence/run decision logic.
<code>app.py</code>	Provides the FastAPI backend, webcam worker, saved reports, and API routes.
<code>frontend/src/App.jsx</code>	Provides the React dashboard for upload, webcam, results, logs, and reports.

The default trained model is stored as:

`models/cheating_bilstm_20260516_180325_all_videos.keras`

The metadata file saved with the model stores the active input settings: 30-frame sequences, sequence stride 15, frame stride 3, 244 feature columns, and tuned suspicious threshold 0.45. The backend reads this metadata so that training, uploaded-video testing, and live webcam testing all use the same structure.

5.5 Identification of Pose and Object

Mediapipe

MediaPipe takes each video frame as input, which is a simple image captured by OpenCV from the exam hall video. It receives this image one by one for every frame of the video. MediaPipe runs three detectors at the same time on each frame. The first one is the Pose detector which finds 33 key points of the full human body like shoulders, arms, and spine to check how the student is sitting or leaning. The second one is the Face Mesh detector which looks at the face and checks whether the face is visible or not, and which direction the head is pointing, like left, right, down, or up. The third one is the Hands detector which detects the position of the hands and checks whether any hand is too close to the face, which can be a sign of whispering or hiding something. As output, MediaPipe gives a set of landmark coordinates and flag values for each frame. These flags tell us things like face is missing, head is turned left, hand is near face, and so on.

All these values are combined into a 244 feature row which is later sent to the Bi-LSTM model for the final decision.

YOLOv8

YOLOv8 takes each video frame as input, which is the same image that OpenCV captures from the video. It looks at the full image at once, not part by part, which makes it very fast. YOLOv8 scans the entire frame in one single look and searches for specific objects that should not normally be present during an exam. It divides the image into a grid and checks every part of the image for suspicious items. If it finds a phone in the frame, it sets the `phone_detected` flag to 1. If it finds a book or copy, it sets the `book_detected` flag to 1. If it finds a chit or small piece of paper, it sets the `chit_detected` flag to 1. If it finds a laptop, it sets the `laptop_detected` flag to 1. If it sees more than one person in the frame, it sets the `extra_person_detected` flag to 1, which means someone else may be helping the student cheat. As output, YOLOv8 gives a set of detection flags for each frame telling whether a suspicious object or extra person was found or not. These flag values are added to the 244 feature row along with the MediaPipe values and sent together to the Bi-LSTM model for the final cheating decision.

5.6 From Landmarks to Behavior Features

The detector output is not used directly as an image or video frame. Instead, every sampled frame is converted into a fixed numerical feature row. This makes the input suitable for machine learning because the Bi-LSTM model cannot understand raw landmark drawings; it needs numerical values that describe the student's posture, movement, and suspicious evidence.

MediaPipe Pose gives 33 body landmarks. For each landmark, the system stores four values: x , y , z , and visibility. The x and y values represent the landmark position in the frame, z gives depth-related position, and visibility indicates how confidently the landmark is detected.

$$33 \times 4 = 132$$

Therefore, 132 pose landmark features are produced from each sampled frame.

Along with static landmark position, movement is also important. A single frame can show where the student is sitting, but it cannot show whether the student is repeatedly moving, turning, bending, or looking away. For this reason, the system calculates motion features by comparing the current frame with the previous processed frame:

$$\Delta x_t = x_t - x_{t-1}, \quad \Delta y_t = y_t - y_{t-1}, \quad \Delta z_t = z_t - z_{t-1}$$

For 33 landmarks, three motion values are stored for each landmark:

$$33 \times 3 = 99$$

These motion features help the model understand changes in body position over time.

In addition to pose landmarks and motion values, the system also stores 13 base behavior and object flags. These include face missing, head left, head right, head down, head up, hand near face, body lean left, body lean right, phone detected, book detected, chit detected, laptop detected, and extra person detected.

The final frame-level feature vector is calculated as:

$$13 + 132 + 99 = 244$$

Thus, each sampled frame is represented by 244 numerical features. These features are later grouped into fixed-length temporal sequences and passed to the Bi-LSTM model. In this way, the system does not make a decision from one frame only; it learns the behaviour pattern across multiple frames.

5.7 Feature Fusion

After MediaPipe and YOLOv8 have done their work on a frame, all of their outputs are combined into a single unified row of 244 values. The order and structure of these 244 values never changes, whether the system is extracting features for training, for a video uploaded by a user, or for a live webcam session. Because the model is trained on rows with this exact structure, any new input must follow the same structure exactly.

Another important aspect of feature fusion is how missing data is handled. Sometimes a frame is blurry or the student is moving quickly, and MediaPipe cannot confidently detect all 33 body landmarks. In these cases, rather than discarding the frame, the missing values are filled with zeros. The model learns to treat these zero-filled rows as neutral rather than suspicious, which prevents poor camera quality from being mistaken for cheating.

Sequence Windowing

A single frame row is not a useful unit for classification. A student who briefly glances sideways is not cheating. A student who looks sideways continuously for 30 seconds might be. The difference lies in what happens over time, not in any single frame.

The active model input is:

$$\mathbf{X}_i \in R^{30 \times 244}$$

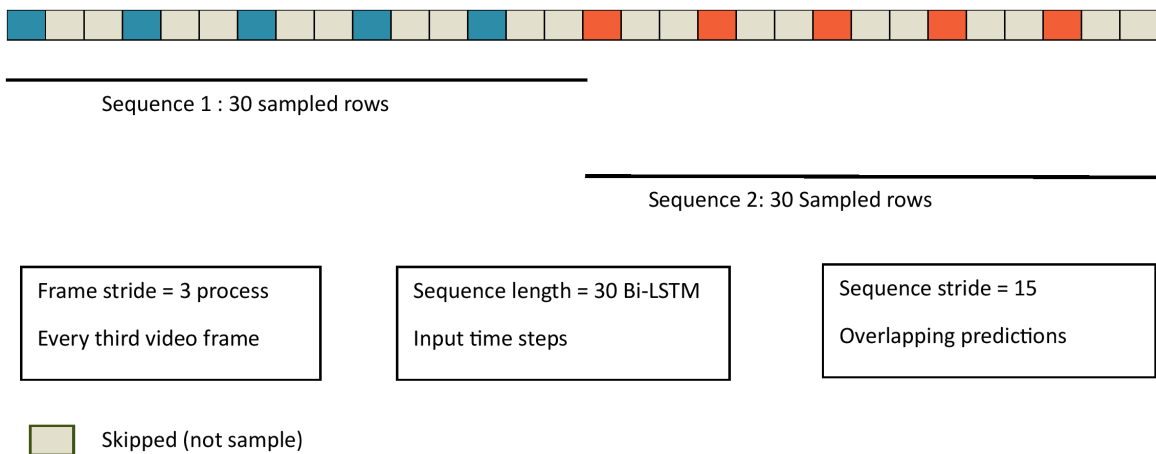


Figure 5.3: Sequence windowing

The sequence builder sorts rows by video and frame number, creates 30-frame windows, and moves forward by 15 sampled frames for the next window. For a window

starting at index s , the sequence is:

$$\mathbf{X}_s = [\mathbf{x}_s, \mathbf{x}_{s+1}, \dots, \mathbf{x}_{s+29}]$$

and the next sequence starts at:

$$s_{\text{next}} = s + 15$$

The sequence label is calculated from the labels inside the window:

$$y_s = \text{round} \left(\frac{1}{30} \sum_{k=s}^{s+29} y_k \right)$$

This overlap means that each piece of footage is seen from two slightly different perspectives, effectively doubling the number of training examples without introducing new data. From the 31,555 frame rows collected across all 102 videos, this windowing process produced 1,951 sequences in total.

5.8 Bi-LSTM Architecture Implementation

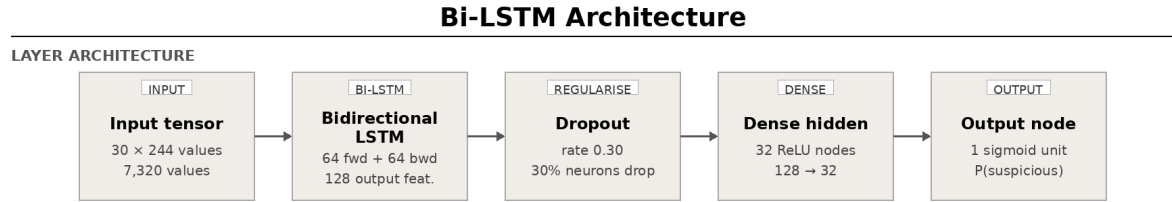


Figure 5.4: Bi-LSTM architecture

The Bi-LSTM model is implemented in `training.py`.

The architecture used in the recommended final model is shown in Table 5.3.

Table 5.3: Implemented Bi-LSTM architecture

Layer	Output Idea	Purpose
Input	30×244	Receives 30 sampled frames with 244 features each.
Bidirectional LSTM	$64 + 64$ units	Learns behaviour patterns in forward and backward temporal directions.
Dropout	0.3	Reduces overfitting after recurrent learning.
Dense ReLU	32 units	Combines learned temporal features into a compact representation.
Dropout	0.2	Adds another regularisation step.
Dense Sigmoid	1 value	Produces suspicious probability between 0 and 1.

The final sigmoid output is:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

where $\hat{y}$ is the model probability for the suspicious class. A value close to 0 means normal behaviour, and a value close to 1 means suspicious behaviour.

5.9 Main System Layers

Table 5.4: Main layers of the implemented system

Layer	Implementation in this Project
Input Layer	Accepts uploaded exam videos through the dashboard and live webcam input through OpenCV.
Feature Layer	Uses <code>CheatingFeatureExtractor</code> to extract pose, face, hand, and object evidence from sampled frames.
Sequence Layer	Uses <code>build_sequences()</code> to create 30-frame windows with stride 15.
Model Layer	Loads the trained Bi-LSTM model and predicts suspicious probability for each sequence.
Decision Layer	Combines model output with rule-based evidence such as phone hits, object counts, repeated side movement, missing face, and repeated hand-near-face behaviour.
Presentation Layer	Uses FastAPI and React to display status, score, sequence timeline, evidence table, charts, and saved reports.

5.10 Training

Training is performed using a function called `train_from_feature_csv()` in `training.py`. The process starts by loading a CSV file that contains features extracted from video frames and checks that all required feature columns are present. The data is then split by video name, not by individual frames. This is important because if frames from the same video appeared in both training and testing, the model would memorize that video instead of learning general patterns. So entire videos are assigned to either training or testing, never both.

The BiLSTM model is then trained only on the training sequences. It reads each sequence twice, once from the first frame to the last and once from the last frame to the first, allowing it to understand the full context of a movement. The model learns by adjusting its weights based on how wrong its predictions were, repeating this process until it improves. After training, a threshold is tuned to decide at what probability score a sequence should be called suspicious. Finally, both the model and its metadata are saved.

5.11 Testing

Testing is done on sequences from videos the model has never seen during training. Each test sequence is passed through the trained BiLSTM, which outputs a probability score. If the score is above the threshold, the sequence is labeled suspicious, otherwise it is labeled normal. The test set contains 376 sequences, with 188 normal and 188 suspicious, ensuring the evaluation is fair and unbiased.

Training is performed through `train_from_feature_csv()` in `training.py`. The implementation first loads the feature CSV, validates that all feature columns exist, builds sequences, splits the data by video name, trains the model, tunes the threshold, evaluates the test set, and saves both the model and metadata.

The actual training setup used for the recommended model is shown in Table 5.5.

Table 5.5: Training configuration used in the implemented model

Setting	Value
Feature CSV	<code>frame_features_20260517_landmark_motion.csv</code>
Total videos	102 videos: 51 normal and 51 suspicious
Sampled feature rows	31,555 rows
Total generated sequences	1,951 sequences
Train sequences	1,575 sequences
Test sequences	376 sequences
Frame stride	3
Sequence length	30 sampled frames
Sequence stride	15 sampled frames
Batch size	16
Maximum epochs	20
Epochs actually run	19
Optimizer	Adam
Loss function	Binary Focal Crossentropy

The test set contains 376 sequences. In the saved metadata, the test support is balanced:

188 normal sequences, 188 suspicious sequences

5.12 Threshold Tuning and Runtime Logic

The model first produces probabilities, not direct final labels. A threshold is needed to convert probability into class label:

$$\hat{y}_{\text{class}} = \begin{cases} 1, & \hat{y} \geq \tau \\ 0, & \hat{y} < \tau \end{cases}$$

Here, τ is the suspicious threshold. The requested threshold was 0.50, but the training code tests thresholds from 0.01 to 0.99 and selects the best threshold using F2-score first, then recall, precision, accuracy, and closeness to 0.50. F2-score is used because recall is very important in suspicious behaviour detection.

The F2-score is:

$$F_2 = \frac{(1 + 2^2) \cdot Precision \cdot Recall}{(2^2 \cdot Precision) + Recall}$$

For the recommended model, the selected threshold is:

$$\tau = 0.45$$

Runtime Decision Logic

The final result is not based only on raw model probability. This is intentional. In exam monitoring, one noisy sequence should not automatically decide the full result. The project therefore combines model output with rule-based evidence.

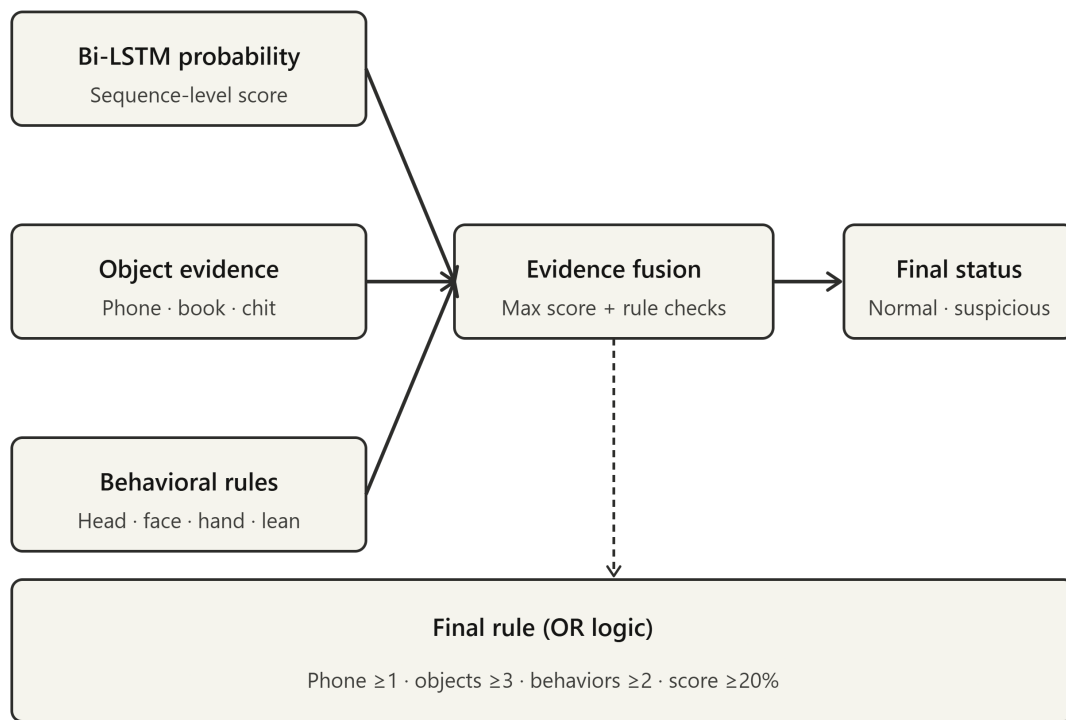


Figure 5.5: Decision Logic: Model Probability + Rule based evidence

The implemented run-level decision can mark a result suspicious when:

- at least one phone detection is found,
- three or more object detections are found,
- at least two behavioural suspicious sequences are found, or
- at least 20% of checked sequences are behaviourally suspicious.

The simplified runtime decision can be written as:

$$\text{FinalSuspicious} = \text{ObjectRule} \vee \text{PhoneRule} \vee \text{BehavioralRule}$$

where:

$$\text{PhoneRule} = \text{phone_detections} \geq 1$$

$$\text{ObjectRule} = \text{object_detections} \geq 3$$

$$\text{BehavioralRule} = (\text{suspicious_behavioral_sequences} \geq 2) \vee \left(\frac{\text{suspicious_behavioral_sequences}}{\text{total_sequences}} \geq 0.20 \right)$$

This rule-based layer is not a replacement for the Bi-LSTM model. It is added so that strong visible evidence, such as phone detection, is not ignored, and so that weak single-frame behaviour does not dominate the result.

5.13 Frontend and Backend

Frontend

The frontend is implemented using React and Vite in:

`frontend/src/App.jsx`

The dashboard has controls for sequence length, sequence stride, frame stride, threshold, YOLO usage, and optional actual label. It supports video upload analysis, live webcam monitoring, saved run browsing, charts, confusion matrix display, and sequence-level evidence tables. The interface is not only for showing a final result — it is also used to inspect why the system reached that result.

Backend

The backend is implemented in `app.py` using FastAPI. It loads the trained model, serves API routes, manages webcam execution, saves prediction runs, and returns report files to the frontend.

Table 5.6: Important backend routes

Route	Purpose
<code>/api/health</code>	Checks backend, model, metadata, and frontend build availability.
<code>/api/model</code>	Returns model metadata, feature columns, and default settings.
<code>/api/analyze-video</code>	Accepts an uploaded video and runs full prediction.
<code>/api/webcam/start</code>	Starts a live webcam test in a background thread.
<code>/api/webcam/status</code>	Returns current live status, score, reason, and sequence log.
<code>/api/webcam/feed</code>	Streams the latest webcam frame to the frontend.
<code>/api/runs</code>	Lists saved prediction runs.
<code>/api/runs/{run_id}/sequence-log</code>	Returns saved sequence-level evidence for a run.
<code>/api/runs/{run_id}/files/{file_key}</code>	Opens saved CSV, HTML, chart, or matrix files.

The backend also protects saved-run file access by checking that requested files remain inside the selected run directory. This avoids returning arbitrary local files through the API.

Saved Output and Reporting

Every completed upload or webcam run is saved under:

`outputs/predictions/runs/`

Each run can contain `sequence_log.csv`, `summary.json`, `report.html`, `metrics.csv`, `confusion_matrix.csv`, and generated SVG charts. This makes the project easier to audit because the final verdict can be checked together with sequence-level probabilities, reasons, object counts, and evaluation metrics when an actual label is supplied.

5.14 Results

To check whether the model is overfitting (memorizing training data instead of learning general patterns), the train and test results were compared. Interestingly, the test accuracy (97.07%) is slightly higher than the training accuracy (92.32%). This does not mean the model has memorized the test videos. Since the dataset was split by video name (not by individual frames), it is possible that the test videos simply contained slightly clearer or easier-to-detect behavior than some training videos.

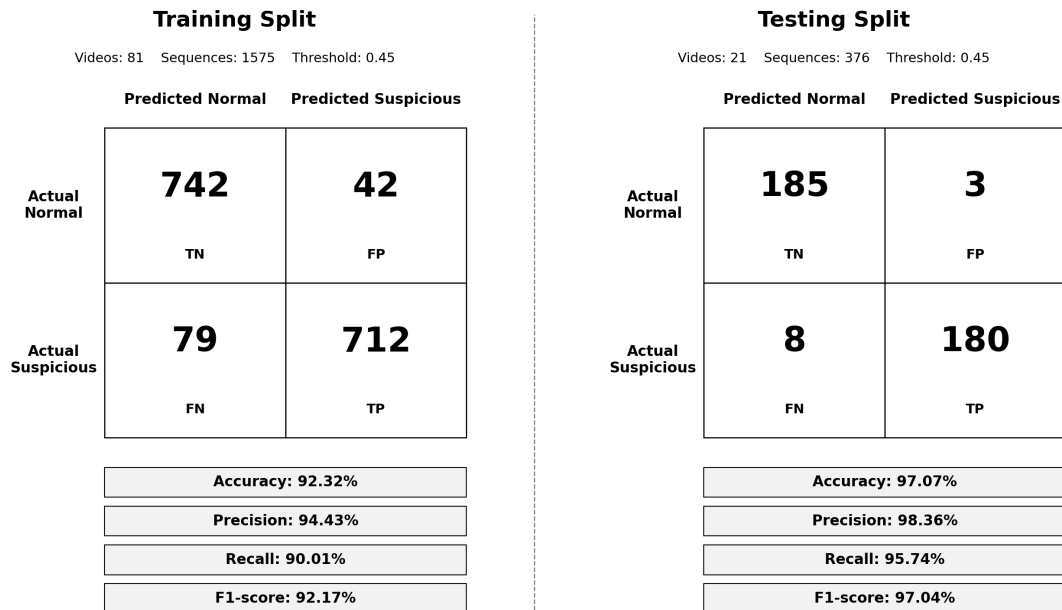


Figure 5.6: Bi-LSTM train test confusion matrices

Table 5.7: Train and Test Comparison

Split	Sequences	Accuracy	Precision	Recall	F1- Score
Train	1,575	92.32%	94.43%	90.01%	92.17%
Test	376	97.07%	98.36%	95.74%	97.04%

Chapter 6

Discussion

Initially, the dataset contained 58 videos, evenly split between 29 normal and 29 suspicious behavior samples. Using this limited data, the proposed Bi-LSTM model achieved an accuracy of about 88%. While it learned basic temporal behavior patterns, the small dataset limited its ability to generalize effectively to unseen data.

To improve performance, the dataset was expanded to 102 balanced videos (51 normal and 51 suspicious), resulting in a larger number of extracted frames and training sequences. With this enhanced dataset, the model's accuracy increased significantly to 97.07%, showing that a larger and more diverse dataset enabled the Bi-LSTM to learn richer behavioral patterns, reduce overfitting, and generalize better.

These results highlight the importance of dataset size and class balance in improving the effectiveness of deep learning-based behavioral analysis systems.

Final Selected Model:

After testing different configurations, the 64-unit Bi-LSTM was chosen as the final model. It was selected because it gives the best overall balance between detecting suspicious behavior, keeping false alarms low, and having a smaller model size suitable for real-time use.

Table 6.1: Final Model Details

Metric	Value
LSTM units (per direction)	64
Total parameters	162,369
Model size	1.90 MB
Decision threshold	0.45
Test sequences	376
Test accuracy	97.07%
precision	98.36%
recall	95.74%
F1-score	97.04%
False alarms	3
Missed suspicious cases	8

Comparison with 128-unit Bi-LSTM:

To make sure the 64-unit model was actually the best choice, a second, larger model with 128 units was also trained and tested under the same conditions.

Table 6.2: 64-unit and 128-unit Bi-LSTM Comparison

Metric	64 Units	128 Units
Parameters	162,369	390,209
Model size	1.90 MB	4.51 MB
Best threshold	0.45	0.56
Test accuracy	97.07%	96.81%
Suspicious precision	98.36%	95.83%
Suspicious recall	95.74%	97.87%
Suspicious F1-score	97.04%	96.84%
False alarms	3	8
Missed suspicious cases	8	4

The 128-unit model catches a few more suspicious cases (missed cases dropped from 8 to 4), but it also raises more false alarms (increased from 3 to 8). Since false accusations during an exam are just as serious as missed cheating, this trade-off matters a lot. Here, compares the resource usage of both models — the number of parameters and the file size. The 64-unit model uses far fewer parameters (162K vs 390K) and is much smaller in size (1.90 MB vs 4.51 MB), making it more practical for real-time deployment. The final comparison shows why the 64-unit model was chosen as the better option overall.

The 128-unit model could still be useful in situations where missing even one suspicious case is more critical than getting an occasional false alarm — for example, in higher-stakes exams where stricter monitoring is preferred.

Sample Prediction Outcomes:

Beyond the train-test results, the system was also tested on a few separate saved videos and live webcam sessions to see how it behaves in practical conditions.

The live webcam run 20260603_012448 was correctly marked suspicious, even though only 2 out of 8 sequences crossed the threshold. Its confusion matrix was:

$$\begin{bmatrix} 0 & 0 \\ 6 & 2 \end{bmatrix}$$

This means that in this short live session, 2 suspicious sequences were correctly detected, but 6 suspicious sequences were missed. This shows that live webcam testing is noticeably harder than testing on pre-recorded videos, since lighting, camera angle, and short recording duration can all affect detection quality.

Table 6.3: Sample prediction outcomes

Input	Final Status		Sequences	Important Outcome
normal_08.mp4	Normal		13	100% normal behavior with no suspicious sequences and no object evidence.
suspicious_phone_10.mp4	Suspicious		25	100% suspicious behavior with strong phone evidence.
20260603_012448 webcam	live	Suspicious	8	2 suspicious behavioral sequences, 25% suspicious ratio, no object detections.
20260603_001039 webcam	live	Normal	6	Actual label was suspicious, but no suspicious sequence crossed the threshold.

The strongest outcome of this project is its ability to combine deep learning-based temporal analysis with explainable rule-based evidence. The Bi-LSTM model captures behavioral patterns over 30-frame sequences, enabling it to understand motion and activity trends across time. In parallel, the rule-based layer enhances interpretability by explicitly highlighting specific suspicious activities such as phone detection, absence of face visibility, repeated side glancing, or hands frequently positioned near the face.

However, the experimental results also indicate certain practical limitations. A high test accuracy does not always guarantee consistent performance in real-world webcam-based environments. External factors such as poor lighting conditions, improper camera angles, short video duration, or occlusions can affect the reliability of predictions. Additionally, the object detection module may occasionally produce false alarms by misclassifying common objects like bottles or books as restricted items.

Overall, the proposed system is best suited as an **exam monitoring support system** rather than a fully autonomous decision-making tool. It assists invigilators by identifying and summarizing suspicious time intervals, providing interpretable reasons for alerts, and maintaining logs for further review. Final decisions, however, should always remain under human supervision to ensure fairness and accuracy.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

The rapid growth of online examinations has increased the need for reliable and automated cheating detection systems. Traditional human-based monitoring can be difficult to perform effectively in remote environments, making it important to develop intelligent systems that can identify suspicious behaviors while maintaining the fairness and integrity of examinations.

To address this challenge, this project developed a complete cheating detection framework using video analysis and deep learning techniques. The system combines OpenCV for video processing, MediaPipe for extracting face, hand, and body pose landmarks, and YOLOv8 for detecting prohibited objects such as mobile phones and books. Rather than making decisions from individual frames, the system analyzes sequences of 30 frames to capture behavioral patterns over time. These extracted features are then provided to a Bi-LSTM model, which is capable of learning temporal relationships and distinguishing between normal and suspicious activities. In addition, a web-based application was developed to support video uploads, live webcam monitoring, report generation, and visualization of results, making the system practical and user-friendly.

The experimental results demonstrate the effectiveness of the proposed approach. The final 64-unit Bi-LSTM model achieved a test accuracy of 97.07%, with a suspicious precision of 98.36% and a suspicious recall of 95.74%. The model correctly classified the majority of normal and suspicious sequences while producing very few false alarms. These results show that the proposed system can accurately detect suspicious examination behavior and can serve as a reliable tool for supporting online exam proctoring.

7.2 Future Work

There are several directions in which this project can be extended and improved

1. Expand the dataset.
2. Attach more annotated object pictures to the model for phone, book, chit, laptop, and negative-background cases.
3. Enhance YOLO confidence calibration to minimize false phone and object alerts.

4. Implement temporal smoothing to prevent one noisy sequence from overpowering the final decision.
5. Implement student identity verification using face recognition, as appropriate for privacy restrictions.
6. Add gaze estimation for more accurate eye-direction analysis.
7. Add audio-based suspicious speech or whispering detection.

Bibliography

- [1] Atoum et al., *Automated Online Exam Proctoring*, 2017.
- [2] Abohamama Hamouda, *A Hybrid Deep Learning Model for Online Exam Proctoring*, 2020.
- [3] Shah, S., Kanchwala, Q., and Mi, H., *Detection of Malpractice in E-exams by Head Pose and Gaze Estimation*, reference paper used for the literature review, available in the project folder `Research_papers_for_reference`.
- [4] A. Abdulkarim, M. Kuliya, and A. B. Adam, *Real-Time AI-Driven Exam Cheating Detection Using YOLO and Pose Estimation: A Multi-Modal Deep Learning Approach*, reference paper used for the literature review, 2025.
- [5] Author(s), *Multiple Instance Learning for Cheating Detection and Localization in Online Examinations*, reference paper used for the literature review, 2024.
- [6] P. Kellnhofer, A. Recasens, S. Stent, W. Matusik, and A. Torralba, *Gaze360: Physically Unconstrained Gaze Estimation in the Wild*, IEEE/CVF International Conference on Computer Vision, 2019.
- [7] Author(s), *Exam Cheating Classification Using a Deep Learning Approach*, International Conference on Artificial Intelligence and Emerging Technology, 2024, doi: 10.1109/GlobalAISummit62156.2024.10947934.
- [8] Author(s), *Cheating Detection in Online Exams Using Transfer Learning*, 5th International Conference on Innovative Research in Applied Science, Engineering and Technology, 2025, doi: 10.1109/IRASET64571.2025.11008131.
- [9] Author(s), *Enhancing Online Exam Security: Deep Learning Algorithms for Cheating Detection*, International Conference on Frontiers of Information Technology, 2023, doi: 10.1109/FIT60620.2023.00032.
- [10] Author(s), *Multi-Modal Online Exam Cheating Detection*, International Conference on Electrical, Computer and Energy Technologies, 2022, doi: 10.1109/ICE-CET55527.2022.9873527.
- [11] Author(s), *Observations of Cheating Behaviours in Online Examinations and Tools for Mitigation*, IEEE Frontiers in Education Conference, 2023, doi: 10.1109/FIE58773.2023.10343363.
- [12] Author(s), *Student Cheating Detection of Academic Integrity Violations Based on Lightweight Deep Convolutional Neural Network and Discrete Fourier Transform*,

- 4th International Conference on Mobile Networks and Wireless Communications, 2024, doi: 10.1109/ICMNWC63764.2024.10872253.
- [13] Author(s), *AI-Driven Detection of Suspicious Activities for Online Exam Integrity: Overview*, International Conference on Meta-Networking, 2025, doi: 10.1109/MEET67398.2025.11335861.
- [14] M. U. H. Al Rasyid, M. R. Mufid, I. G. A. Negara, R. N. Baiti, I. G. A. W. K. Dewi, N. M. S. Rani, and N. P. I. Yuliana, reference work on CNN, eye gaze detection, head pose detection, and lip movement detection, in *Proceedings of the International Conference on Applied Science and Technology on Engineering Science 2025*, doi: 10.2991/978-94-6463-926-1_97.
- [15] OpenCV Documentation, *Open Source Computer Vision Library*, [online]. Available: <https://opencv.org>.
- [16] Google MediaPipe Documentation, *Pose, Face Mesh, and Hands solutions*, [online]. Available: <https://developers.google.com/mediapipe>.
- [17] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, *OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 2021.
- [18] Ultralytics YOLOv8 Documentation, [online]. Available: <https://docs.ultralytics.com>.
- [19] TensorFlow and Keras Documentation, [online]. Available: <https://www.tensorflow.org>.
- [20] scikit-learn Documentation, [online]. Available: <https://scikit-learn.org>.
- [21] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, 1997.
- [22] M. Schuster and K. K. Paliwal, “Bidirectional Recurrent Neural Networks,” IEEE Transactions on Signal Processing, 1997.
- [23] T. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, “Focal Loss for Dense Object Detection,” ICCV, 2017.
- [24] W. M. Rahmawati, A. S. Ahsan, D. S. Santoso, R. Edelani, and M. Latif, “Cheating Detection System Based on Eye Gaze, Head Pose and Lip Movement Detection Using CNN,” in *Proceedings of iCAST-ES 2025*, pp. 869–876, 2025.
- [25] A. Abdulkarim, M. Kuliya, and A. B. Adam, “Real-Time AI-Driven Exam Cheating Detection Using YOLO and Pose Estimation: A Multi-Modal Deep Learning Approach,” *IRE Journals*, vol. 9, no. 6, pp. 847–856, 2025.
- [26] A. Abozaid and A. Atia, “Multi-Modal Online Exam Cheating Detection,” in *International Conference on Electrical, Computer and Energy Technologies (ICECET)*, pp. 1–6, 2022.

- [27] H. Abourifa, I. Qarbal, S. Ouahabi, N. Sael, S. Elfilali, and K. El Guemmat, "Cheating Detection in Online Exams Using Transfer Learning," in *5th International Conference on Innovative Research in Applied Science, Engineering and Technology (IRASET)*, pp. 1–6, 2025.
- [28] S. Lamba and N. Sharma, "Deep Learning-Based Multimodal Cheating Detection in Online Proctored Exams," *Journal of Electrical Systems*, vol. 20, no. 3, pp. 7375–7383, 2024.
- [29] Y. Zhang and L. Wang, "Video-Based Examination Surveillance Systems for Academic Integrity," *International Journal of Computer Vision Applications*, vol. 12, no. 4, pp. 210–220, 2020.
- [30] X. Cheng, Y. Liu, and H. Wang, "Automated Online Examination Monitoring Using Computer Vision Techniques," *Journal of Educational Technology Systems*, vol. 50, no. 2, pp. 145–158, 2021.
- [31] M. Hossain, A. Rahman, and S. Islam, "Visual Behaviour Analysis Framework for Online Examination Monitoring," *IEEE Access*, vol. 9, pp. 112345–112356, 2021.
- [32] F. Kamalov, H. Sulieman, and D. S. Calonge, "AI-Based Proctoring Systems Using Facial Behaviour Analysis," *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 8, pp. 520–528, 2021.
- [33] A. Abozaid and A. Atia, "Multi-Modal Online Exam Cheating Detection," in *International Conference on Electrical, Computer and Energy Technologies (ICECET)*, pp. 1–6, 2022.
- [34] M. Asif, A. Khan, and Others, "Enhancing Online Exam Security: Deep Learning Algorithms for Cheating Detection," in *International Conference on Frontiers of Information Technology (FIT)*, pp. 126–131, 2023.
- [35] M. E. Bishaw, S. Vats, A. O. Salau, K. Kaushik, W. B. Alemu, and T. T. Tin, "Exam Cheating Classification Using a Deep Learning Approach," in *International Conference on Artificial Intelligence and Emerging Technology (Global AI Summit)*, pp. 1010–1015, 2024.
- [36] A. Shiri, M. Hosseini, and R. Ahmadi, "Comparative Analysis of Deep Learning Models for Online Exam Cheating Detection," *Journal of Artificial Intelligence Research*, vol. 78, pp. 455–470, 2024.
- [37] A. Abdulkarim, M. Kuliya, and A. B. Adam, "Real-Time AI-Driven Exam Cheating Detection Using YOLO and Pose Estimation: A Multi-Modal Deep Learning Approach," *Iconic Research and Engineering Journals*, vol. 9, no. 6, pp. 847–856, 2025.
- [38] H. Abourifa, I. Qarbal, S. Ouahabi, N. Sael, S. Elfilali, and K. El Guemmat, "Cheating Detection in Online Exams Using Transfer Learning," in *5th International Conference on Innovative Research in Applied Science, Engineering and Technology (IRASET)*, pp. 1–6, 2025.

- [39] W. M. Rahmawati, A. S. Ahsan, D. S. Santoso, R. Edelani, and M. Latif, “Cheating Detection System Based on Eye Gaze, Head Pose and Lip Movement Detection Using CNN,” in *Proceedings of the International Conference on Applied Science and Technology on Engineering Science (iCAST-ES)*, pp. 868–875, 2025.
- [40] M. Dahan, Z. Mohamad, S. Safei, A. Alhammadi, and J. K. Yousafzai, “AI-Driven Detection of Suspicious Activities for Online Exam Integrity: Overview,” in *International Conference on Meta-Networking (MEET)*, pp. 1–7, 2025.

Appendix A

Software and Hardware Requirements

A.1 Software Requirements

Table A.1: Software requirements

Software	Purpose
Python 3.10/3.11	Backend, model training, inference
OpenCV	Video frame processing
MediaPipe	Pose, face, and hand landmark extraction
Ultralytics YOLOv8	Object detection
TensorFlow/Keras	Bi-LSTM model training and prediction
NumPy/Pandas	Numerical processing and CSV handling
scikit-learn	Metrics, train-test split, confusion matrix
FastAPI	Backend API
React + Vite	Frontend dashboard

A.2 Hardware Requirements

- Laptop or desktop system with webcam.
- Minimum 8 GB RAM recommended.
- CPU-based inference is possible.
- GPU is useful for faster YOLO and TensorFlow training.

Appendix B

Important Code Snippets

B.1 Bi-LSTM Model

```
def build_bilstm_model(sequence_length: int, num_features: int,
    ↪ lstm_units: int = 64):
    from tensorflow.keras import Input
    from tensorflow.keras.layers import Bidirectional, Dense,
        ↪ Dropout, LSTM
    from tensorflow.keras.losses import BinaryFocalCrossentropy
    from tensorflow.keras.models import Sequential

    focal_loss = BinaryFocalCrossentropy(
        apply_class_balancing=True,
        alpha=0.65,
        gamma=2.0,
    )
    model = Sequential([
        Input(shape=(sequence_length, num_features)),
        Bidirectional(LSTM(lstm_units)),
        Dropout(0.3),
        Dense(32, activation="relu"),
        Dropout(0.2),
        Dense(1, activation="sigmoid"),
    ])
    model.compile(optimizer="adam", loss=focal_loss, metrics=["
        ↪ accuracy"])
    return model
```

Listing B.1: Bi-LSTM model architecture

B.2 Final Decision Rule

```
final_suspicious = (
    phone_detections >= 1
    or object_detections >= 3
    or behavioral_suspicious_sequences >= 2
    or behavioral_suspicious_ratio >= 0.20
)
```

Appendix C

Project Commands

```
python scripts/extract_features.py --dataset dataset --output  
    ↪ outputs/features/frame_features_20260517_landmark_motion.  
    ↪ csv --frame-stride 3
```

Listing C.1: Feature extraction command

```
python scripts/train_model.py --features outputs/features/  
    ↪ frame_features_20260517_landmark_motion.csv --model models/  
    ↪ cheating_bilstm_20260516_180325_all_videos.keras --  
    ↪ threshold 0.5
```

Listing C.2: Training command

```
python scripts/predict_video.py --video path\to\exam_video.mp4 --  
    ↪ model models/cheating_bilstm_20260516_180325_all_videos.  
    ↪ keras
```

Listing C.3: Prediction command

```
python app.py  
cd frontend  
npm run dev
```

Listing C.4: Run backend and dashboard